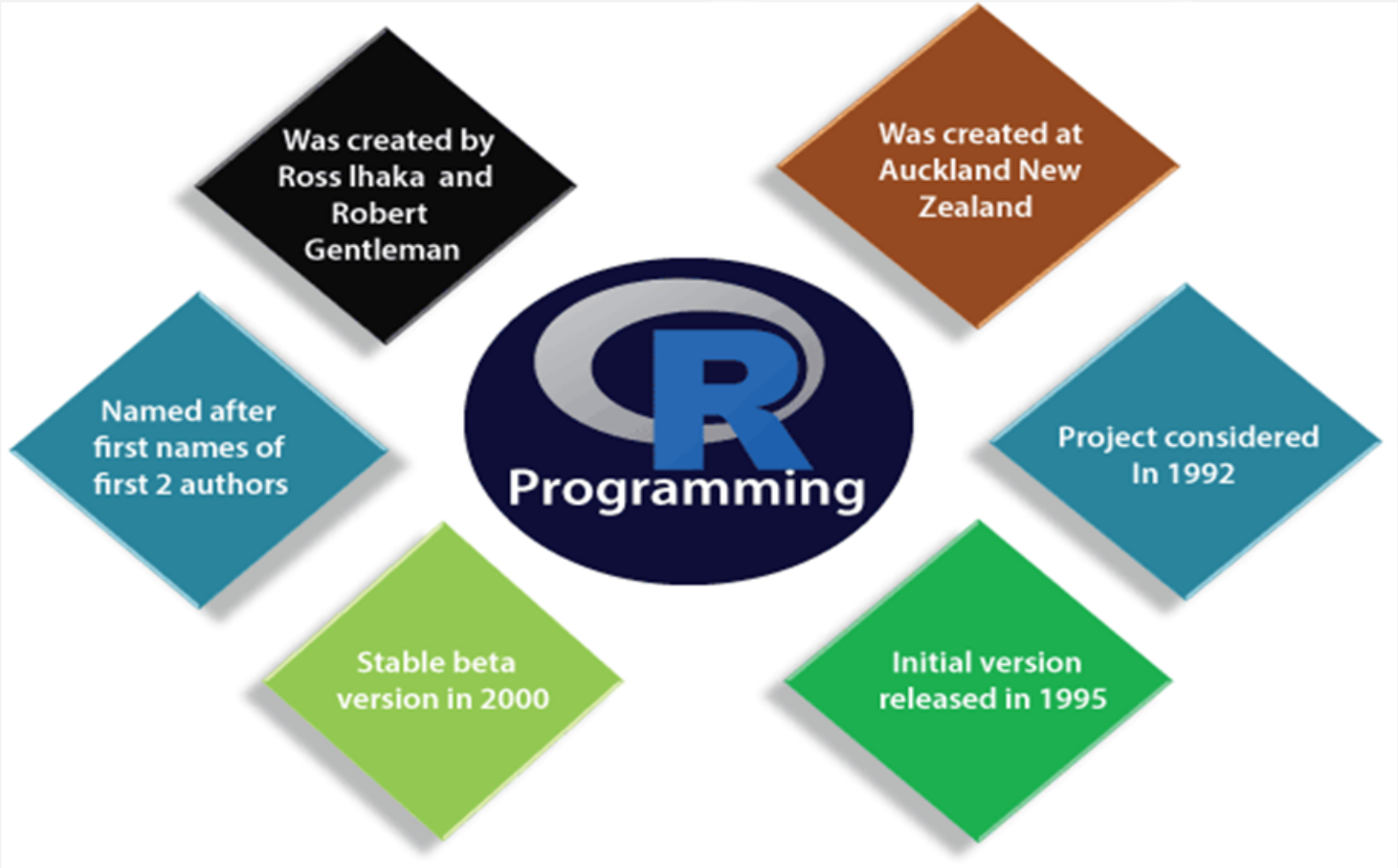
R Programming-Basics

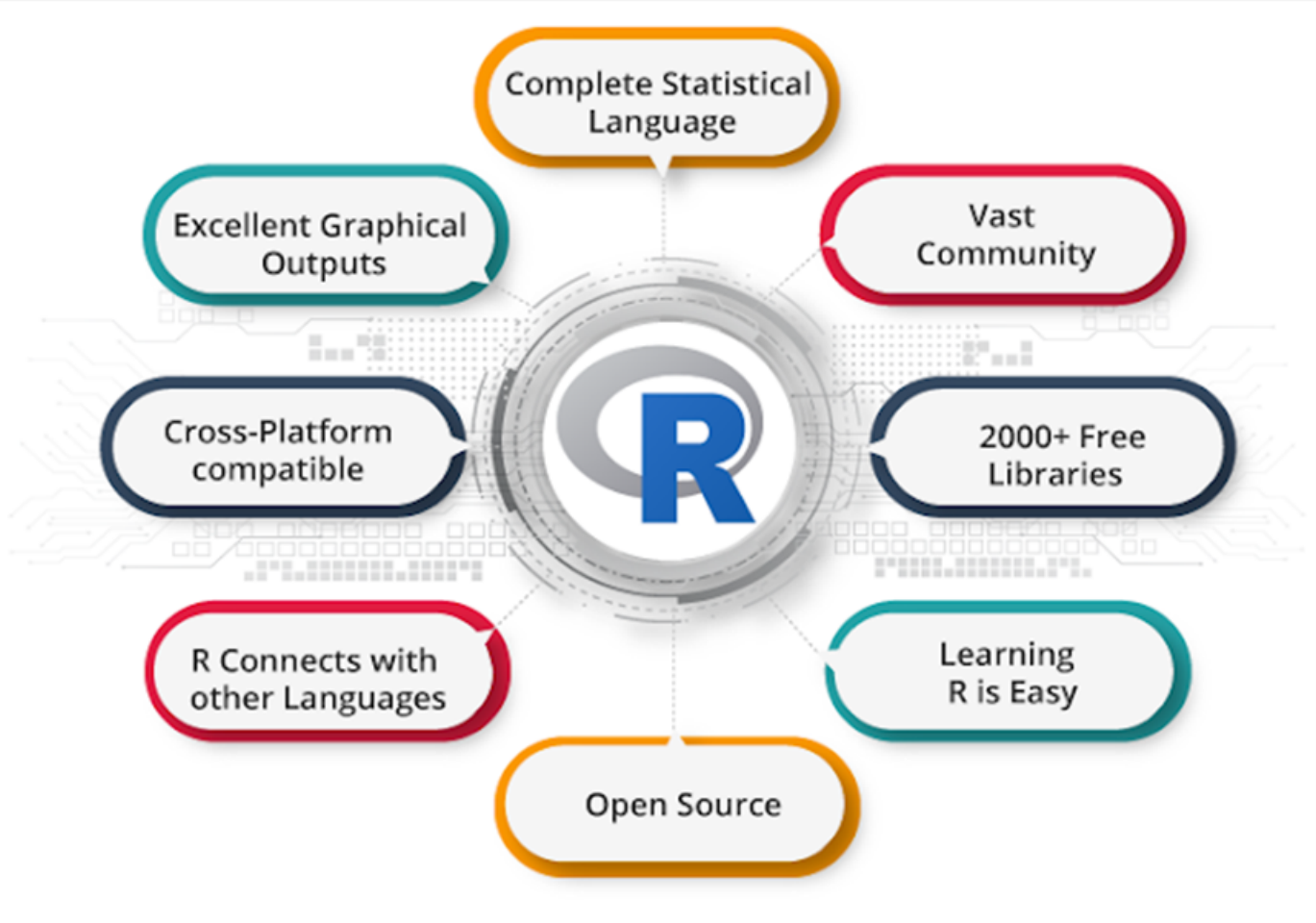
Objective

- Introduction to R
- Installation of R
- Keywords
- Identifier
- Variables
- Data Types
- Operators
- Input/output
- Your First R Program
- Comments
- workspace in r

Introduction to R



Features of R



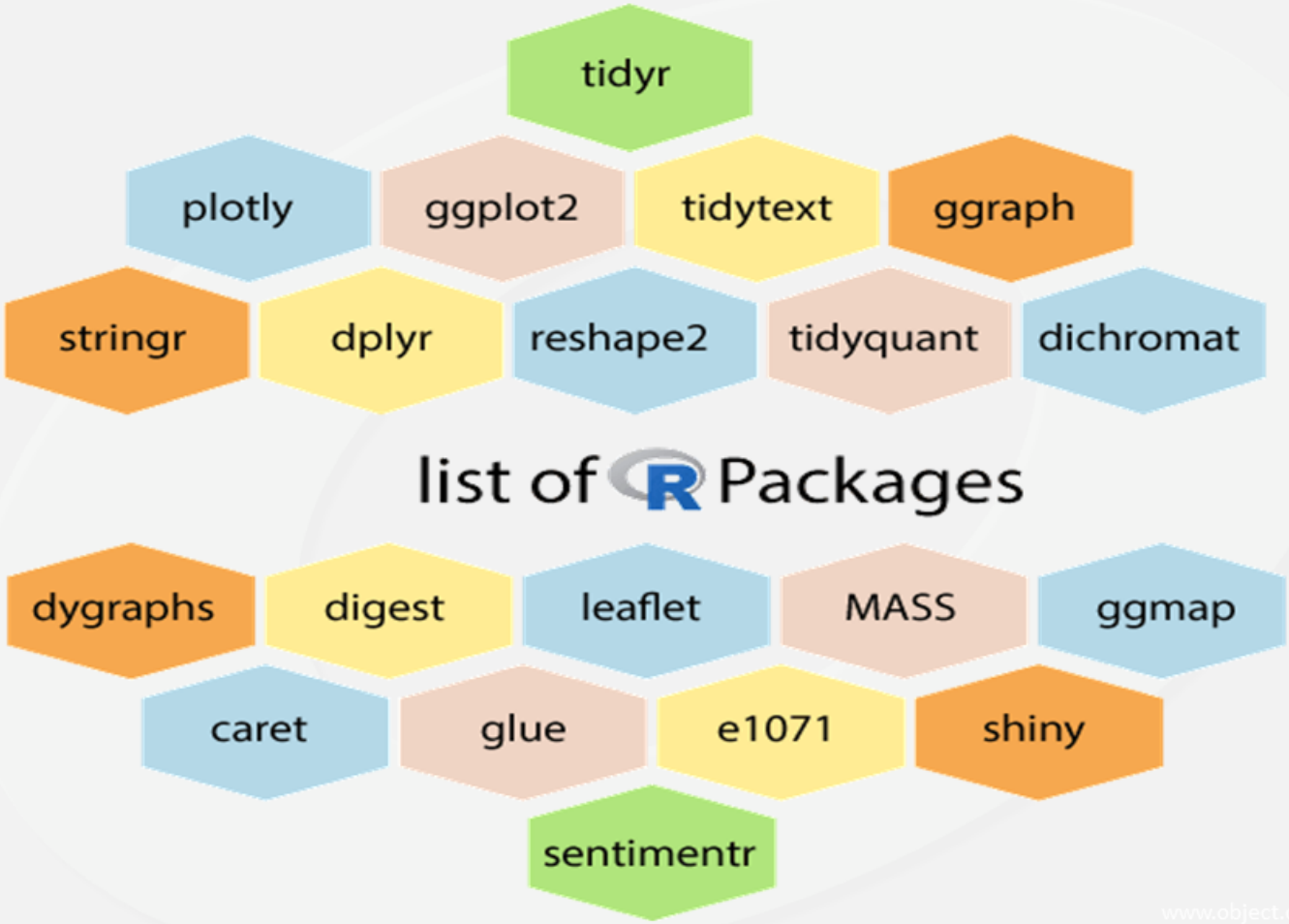
www.object.co.in

Diff Between Interpreter and Compiler

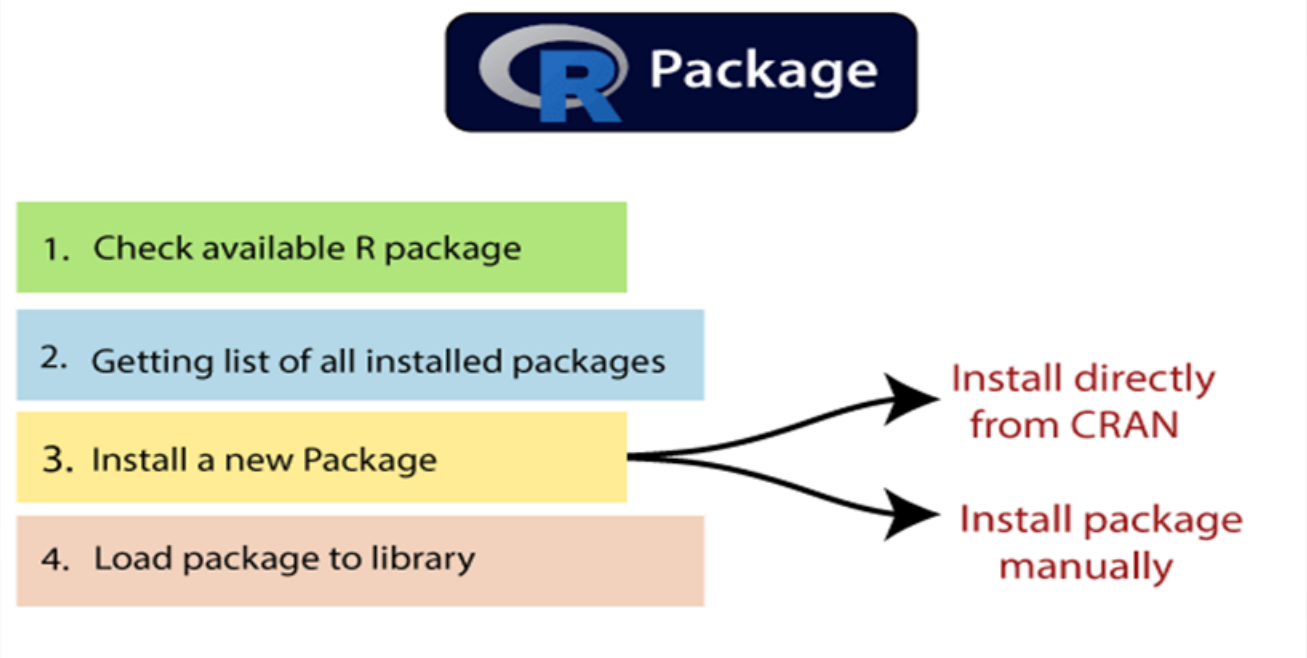
OBJECT™ TECHNOLOGIES

Python	R
Python is a programming language that works quickly and integrates with systems effectively.	R is a language that is specifically designed for statistical analysis.
Python has significantly more flexibility in interacting with other programming languages and is quite fast compared to R. It is a general-purpose programming language and therefore is more flexible to combine with other languages such as Java and PHP.	R is significantly slower than Python and is less adept at integrating with other programming languages. Whereas R is more powerful when it comes to statistical analysis
Community - Python.org.	Community - R-bloggers.
Users: Engineers, programmers, web developers.	Users: Engineers, researchers, statisticians
Data wrangling, machine learning, preprocessing, text mining, web scraping. Python has the more capabilities over R when it comes to creating a general-purpose program that interacts more seamlessly with other programming languages.	Statistics, regression analysis, time series analysis, visualization. R has slower run time than Python, and less flexibility in interacting with other languages. However, R is significantly more ideal for statistical implemention before using other languages.

R packages



www.object.co.in



The "Comprehensive R Archive Network" (CRAN) is **a collection of sites which carry identical material, consisting of the R distribution(s), the contributed extensions, documentation for R, and binaries**.

R provides **library**() function, which allows us to get the list of all the installed packages.

Installation of R

- **Step 1:**
  The official site https://cloud.r-project.org provides binary files for major operating systems including Windows, Linux, and Mac OS.

```
Go to r cmd prompt ->  R.version
```

Step 2:

Install R Studio

https://posit.co/download/rstudio-desktop/

www.object.co.in

R IDE

- **R Studio**
- **Atom**
- **Visual Studio Code**
- **JupyterLab**
- **emacs**
- **Aquamacs**
- **Sublime Text 3**.

However, RStudio is probably the most common and beginner-friendly IDE for R Users, so we'll focus on RStudio.

www.object.co.in

R IDE

RStudio

File Edit Code View Plots Session Build Debug Tools Help

Untitled1 | naivebayes.R*

```r
telecom <- read.csv("D:\\Statistics\\NB_k_NN\\Telecom.csv")

library(caret)

set.seed(333)
intrain <- createDataPartition(y=telecom$Response,p=0.7,list = FALSE)

training <- telecom[intrain,   ]
testing <- telecom[-intrain,]

library(e1071)
classifier <- naiveBayes(training[,1:2], training[,3])

PredY <- predict(classifier, newdata=testing[,-3], type="class")
PredYProb <- predict(classifier, newdata=testing[,-3],type="raw")

PredY <- factor(PredY,levels = c("Y","N"))
testing[,3] <- factor(testing[,3],levels = c("Y","N"))

# PredYRaw <- predict(classifier, newdata=testing[,-3], type="raw")

tbl <- table(predY, testing[,3],dnn=list('predicted', 'actual'))
```

Environment | History

Global Environment

Data

intrain	int [1:106, 1] 2 4 5 6 7 8 10 12 13 15 ...
PredYProb	num [1:44, 1:2] 0.0556 0.9424 0.4961 0.0556...
telecom	150 obs. of 3 variables
testing	44 obs. of 3 variables
training	106 obs. of 3 variables

Values

classifier	List of 4
PredY	Factor w/ 2 levels "Y","N": 1 2 1 1 1 1 1 2 1...
tbl	'table' int [1:2, 1:2] 22 1 8 13

Console

```
Mcnemar's Test P-Value : 0.0455003

            Sensitivity : 0.9565
            Specificity : 0.6190
         Pos Pred Value : 0.7333
         Neg Pred Value : 0.9286
             Prevalence : 0.5227
         Detection Rate : 0.5000
   Detection Prevalence : 0.6818
      Balanced Accuracy : 0.7878

       'Positive' Class : Y
```

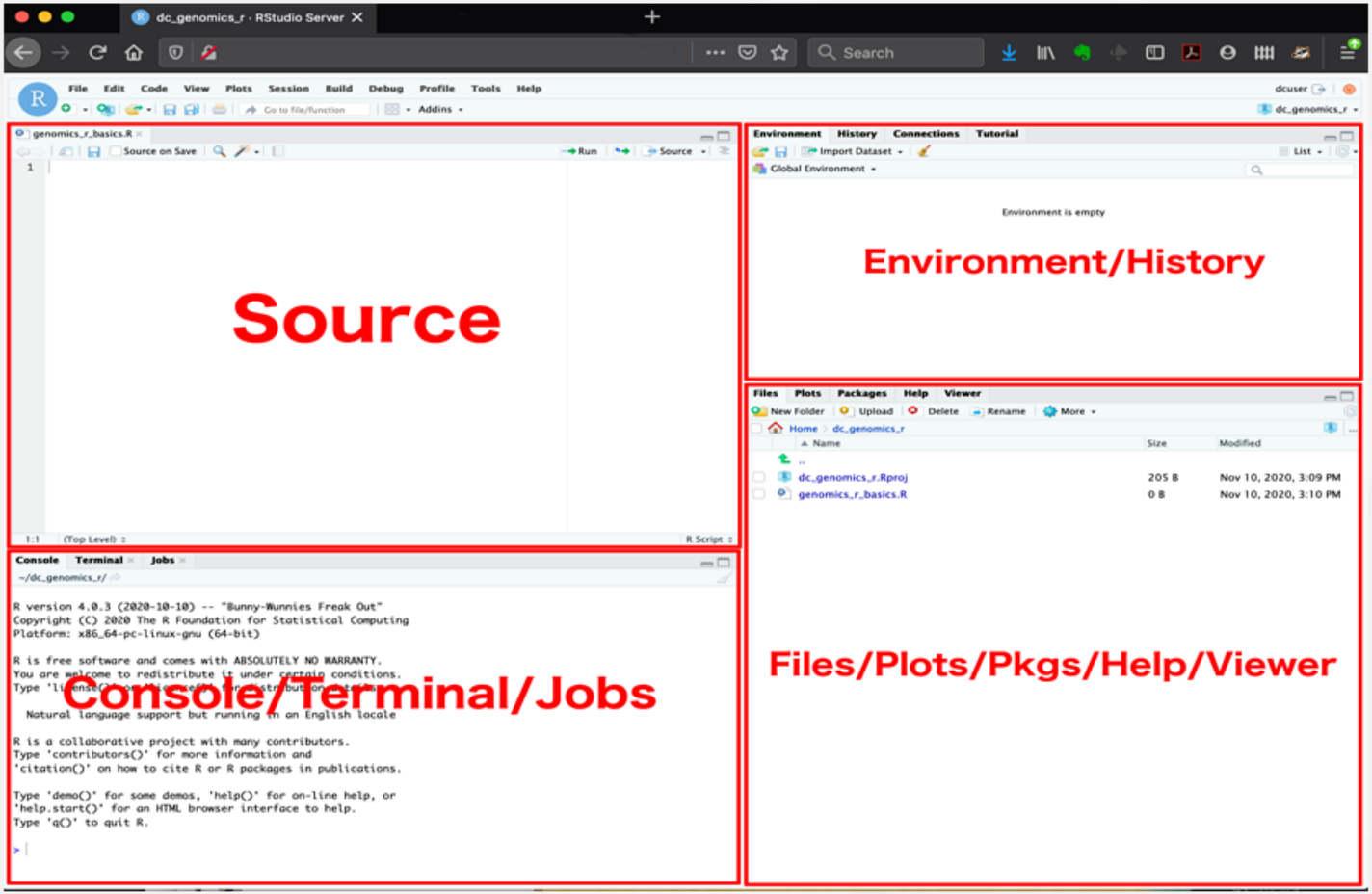
Files | Plots | Packages | Help | Viewer

Install | Update

Name	Description	Version
User Library		
acepack	ace() and avas() for selecting regression transformations	1.3-3.3
arules	Mining Association Rules and Frequent Itemsets	1.3-1
arulesViz	Visualizing Association Rules and Frequent Itemsets	1.1-0
assertthat	Easy pre and post assertions.	0.1
automap	Automatic interpolation package	1.0-14
BH	Boost C++ Header Files	1.58.0-1
bitops	Bitwise Operations	1.0-6
car	Companion to Applied Regression	2.1-0
caret	Classification and Regression Training	6.0-58
caTools	Tools: moving window statistics, GIF, Base64, ROC AUC, etc.	1.17.1

www.object.co.in

R Studio Information



Keywords

- R Keywords are special reserved words that convey a special meaning . Keywords are also called as "reserved names.
- These keywords **can't** be used as a variable.
- **?reserved** or **help(reserved) will give you Keywords.**
- **Following are keywords in R Programming.**

Reserved words in R

if	else	while	repeat	for
function	in	next	break	TRUE
FALSE	NULL	Inf	NaN	NA
NA_integer_	NA_real_	NA_complex_	NA_character_	…

Identifier

An identifier is a name given to entities like class, functions, variables, etc. It helps to differentiate one entity from another.

Rules for writing identifiers in R:

- Identifiers can be a combination of letters, digits, period (.), and underscore (_).
- It must start with a letter or a period. If it starts with a period, it cannot be followed by a digit.
- Reserved words in R cannot be used as identifiers.

Valid Variables	Invalid Variables
var_name2.	var_name%
.var_name	2var_name
var.name	.2var_name
	_var_name

Variables

- Variables are containers/reserved memory location for storing values.
- R Language Variables are not declared with a data type rather they take the data type of the R-object assigned to them.
- variable names are case-sensitive and they follow identifier rules.

R supports three ways of variable assignment:

1. **Using equal operator- data is copied from right to left.**
2. **Using leftward operator- data is copied from right to left.**
3. **Using rightward operator- data is copied from left to right.**

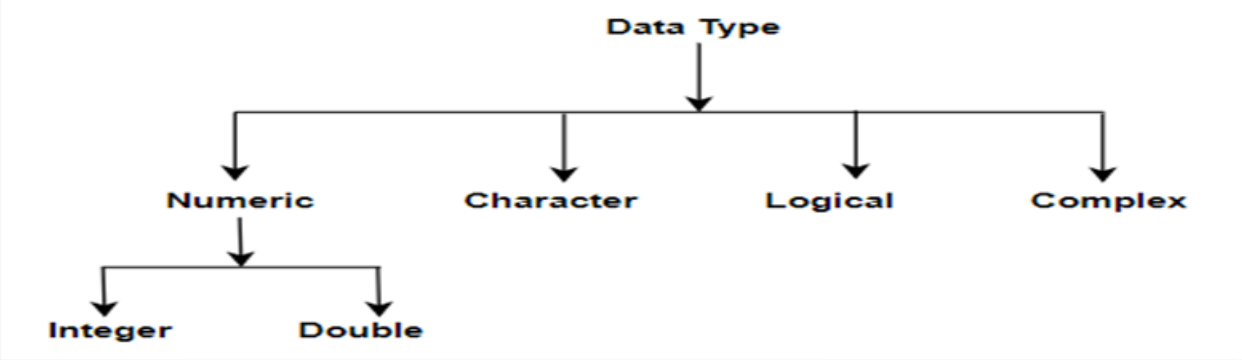
```
#using equal to operator
variable_name = value
#using leftward operator
variable_name <- value
#using rightward operator
value -> variable_name
```

class() function
This built-in function is used to determine the data type of the variable provided to it.

```
var1 = "hello"
print(class(var1))
```

Data Types

- Basic data types in R can be divided into the following types:
- numeric - (10.5, 55, 787)
- integer - (1L, 55L, 100L, where the letter "L" declares this as an integer)
- complex - (9 + 3i, where "i" is the imaginary part)
- character (string) - ("k", "R is exciting", "FALSE", "11.5")
- logical (boolean) - (TRUE or FALSE)



Operators

Arithmetic Operators	+	-	*	/	%%	%/%	^
Relational Operators	<	>	==	<=	>=	!=	
Logical Operators	&	|	!	&&	||		
Assignment Operators	=	<-	->	<<-	->>		
Misc. Operators	:	%in%	%*%				

www.tutorialkart.com

www.object.co.in

Miscellaneous Operators

- colon operator – It is used to generate a series of numbers in sequence for a vector.
  For example:
  x <- 0:9 print(x)
  Output:[1] 0 1 2 3 4 5 6 7 8 9
- %in% :- This operator is used to check if an element belongs to a vector or not.For example:
  v1 <- 5
  t <- 0:9
  print(v1 %in% t)
  Output:[1] TRUE
- %* %: This operator multiplies a matrix with its transpose.
  For example:
- M = matrix(c(1,2,3,4), nrow = 2,ncol = 2,byrow = TRUE)
  T= M %*% t(M)
  print(T)
  Output: [,1] [,2]
  [1,] 5 11
  [2,] 11 25

www.object.co.in

Operators

R has multiple operators that allow you to perform a variety of tasks. Arithmetic operators let you perform arithmetic such as addition and multiplication. Relational operators are used to compare between values. Logical operators are used for Boolean operators.

Arithmetic Operators

```
a + b #Sums two variables
a - b #Subtracts two variables
a * b #Multiply two variables
a / b #Divide two variables
a ^ b #Exponentiation of a variable
a%%b  #Remainder of a variable
a%/%b #Integer division of variables
```

Relational Operators

```
a == b #Tests for equality
a != b #Tests for inequality
a > b #Tests for greater than
a < b  #Tests for lower than
a >= b #Tests for greater than or equal to
a <= b #Tests for less than or equal to
```

Logical Operators

```
!  #Logical NOT
&  #Element-wise logical AND
&& #Logical AND
|  #Element-wise logical OR
|| #Logical OR
```

Assignment Operators

```
x <- 1 # Assigns a variable to x
x = 1 #Assigns a variable to x
```

Other Operators

```
%in% #Identifies whether an element belongs to a vector
$ #Allows you to access objects stored within an object
%>% #Part of magrittr package, it's used to pass objects to functions
```

R Input Methods

- Using **readline()** method
- Using **scan() method**

as.integer(n); —> convert to integer
as.numeric(n); —> convert to numeric type (float, double etc)
as.complex(n); —> convert to complex number (i.e 3+2i)
as.Date(n) —> convert to date …, etc

Syntax:
var1 = readline(prompt = "Enter any number : ");
var1 = readline("Enter any number : ");
x = scan(what = double()) —-for double
x = scan(what = " ") —-for string
x = scan(what = character()) —-for character

www.object.co.in

R output method

- In R there are various methods to print the output.
- **print()** is most common method to print output in R program.n called print() is used.
  print("any string") or, print(variable)
- no need to use any function for print the output if program is written on console.
- Print output using **sprintf()** function
  sprintf("any string %d", variable) or, sprintf("any string %s", variable) or, sprintf("any string %f", variable)) etc.
- Print output using **cat()** function
  Syntax: cat("any string") or, cat("any string", variable)
- Print output using **message()** function
  message("any string") or, message("any string", variable)
- Print output using **paste()** function inside print() function
  print(paste("any string", variable)) or, print(paste0(variable, "any string"))
  The difference between **paste() and paste0()** is that the argument sep by default is " "(paste) and ""(paste0).

www.object.co.in

First R Program

OBJECT™ TECHNOLOGIES

R provides us the two ways to run a program:

1. Using Interactive interpreter prompt
2. Using a script file

1. Interactive interpreter prompt

R provides us the feature to execute the R statement one by one at the interactive prompt.

To open the interactive mode, open the terminal (or command prompt) and type **R**

2.Using a script file

We need to write our code into a file which can be executed later. For this purpose, open an editor like notepad, create a file named first.r (R used .r extension) and write the following code in it.

```
print ("hello world");
```

open cmd prompt type - rscript first.r

either open cmd from current folder or give full path of file.

www.object.co.in

Comments

Single-Line Comments in R

hash symbol # to write a single-line comment.

Example 1:

```
# printing a string
print('Hello world')

Output
Hello world
```

Getting Help:

?mean Get help of a particular function.

help.search('weighted mean') Search the help files for a word or phrase.

help(package = 'dplyr') Find help for a package.

www.object.co.in

workspace

- R Workspace
- R workspace is your current R working environment and includes all the objects and functions you have created.
- The user can anytime save an image of the current workspace that is automatically reloaded the next time R is started.

Workspace Objects

- Viewing list of objects in workspace

```
> ls()
[1] "a" "b" "c" "d" "f" "s"
```

- Saving workspace with name “first”

```
> save.image("D:\\first")
```

- Loading workspace with name “first”

```
> load("D:\\first")
```


www.object.co.in

Summary

- Introduction to R
- Installation of R
- Keywords
- Identifier
- Variables
- Data Types
- Operators
- Input/output
- Your First R Program
- Comments

www.object.co.in

OBJECT™ TECHNOLOGIES

Thank You